



25 September 2025 | M2 SAR | Paris, France

Temporal Logics

M2 SAR - UE Applications et systèmes temps réel répartis embarqués

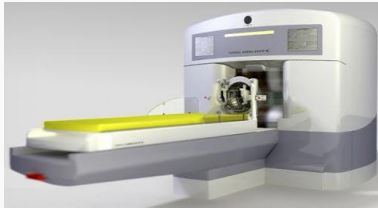
Dylan Marinho

Sorbonne Université, CNRS UMR 7606, LIP6



Context: Verifying complex timed systems

- ▶ **Critical** systems: Failures may result in **dramatic** consequences
- ▶ Need for early bug detection
 - ▶ Bugs discovered when final testing: **expensive**
 - ▶ Need for a thorough **specification** and **verification** phase



Therac-25
(USA, 1980s)



MIM-104 Pat. Mis. Fail.
(Iraq, 1991)



Sleipner A offshore platform
(Norway, 1991)



Ariane flight V88
(France, 1996)

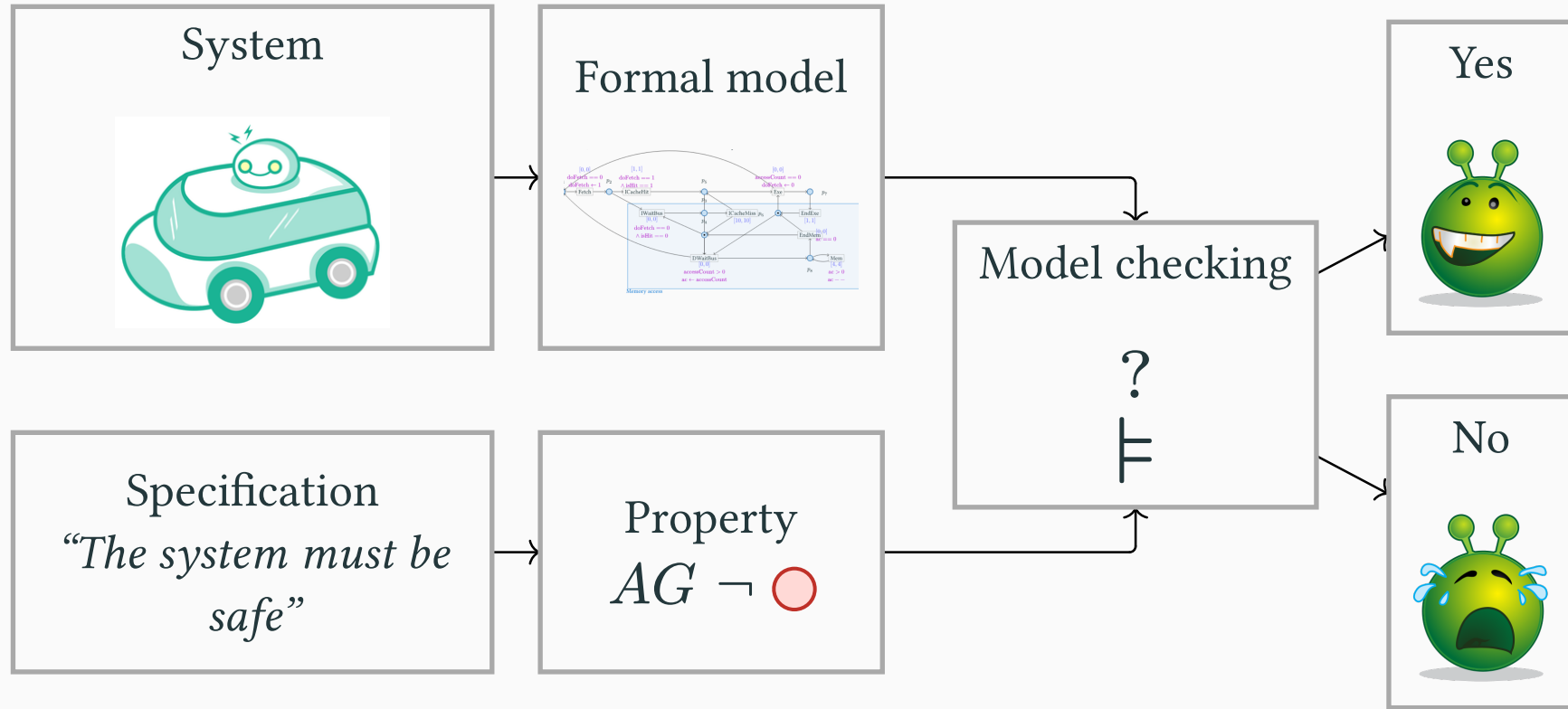
- ▶ Verification is needed to ensure the absence of bugs

- ▶ We want **formal guarantees** → formal methods

- ▶ Various methods:
 - ▶ Abstract interpretation
 - ▶ Static analysis
 - ▶ **Model checking**
 - ▶ Theorem proving



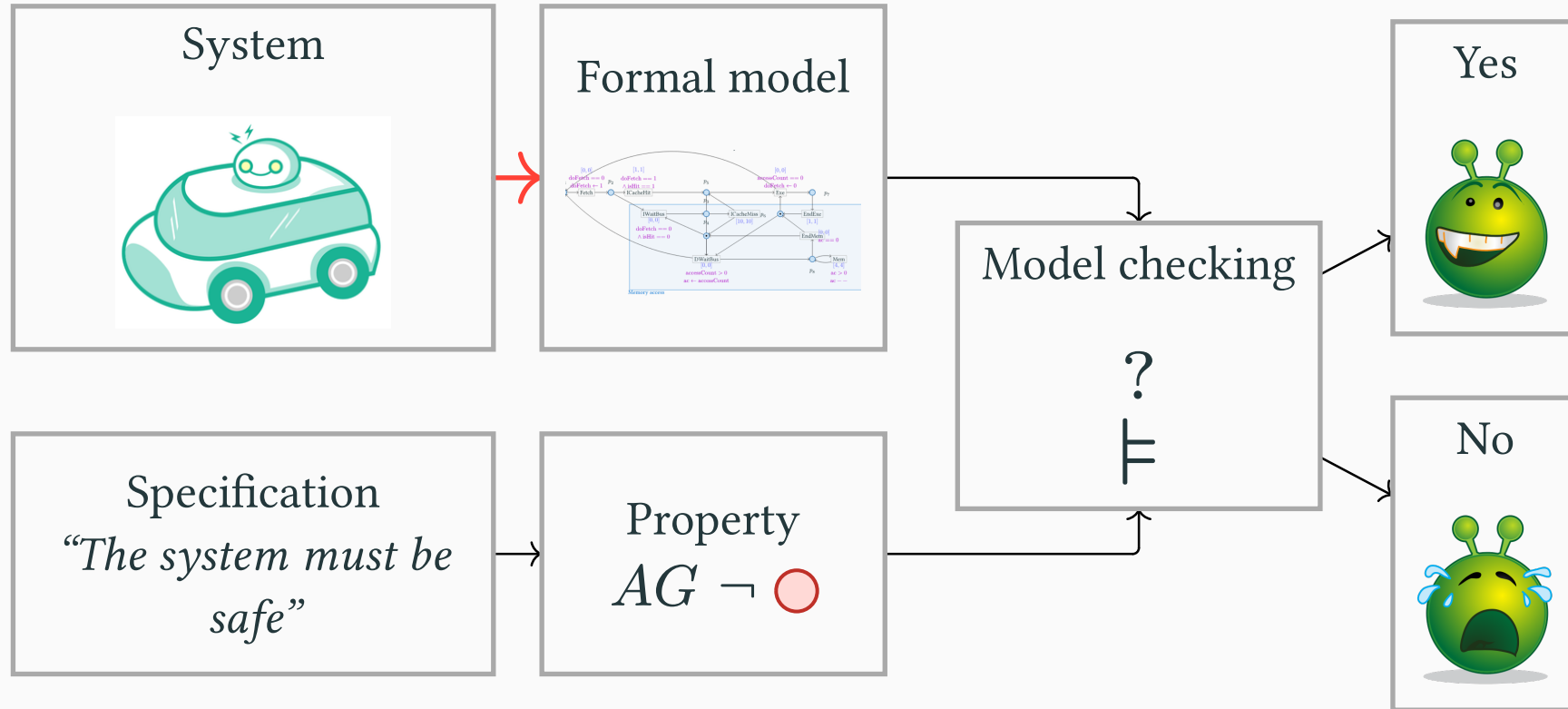
Context: Model checking overview



Question: does the model of the system **satisfy** the property?

Modeling

Modeling



- ▶ We want to verify the behavior of the system **over time**
 - ▶ Notion of **state** at a given instant
 - ▶ System actions → **change** of state
- ▶ Additional information about
 - ▶ communication (notion of **action**)
 - ▶ properties verified by states (**atomic propositions**)

Definition 1

A finite automaton is a quintuplet $(Q, \Sigma, q_0, F, \longrightarrow)$ with

- ▶ a set of states Q
- ▶ an alphabet of actions Σ
- ▶ an initial state $q_0 \in Q$
- ▶ final states $F \subseteq Q$
- ▶ a transition relation $\longrightarrow \subseteq Q \times (\Sigma \cup \varepsilon) \times Q$

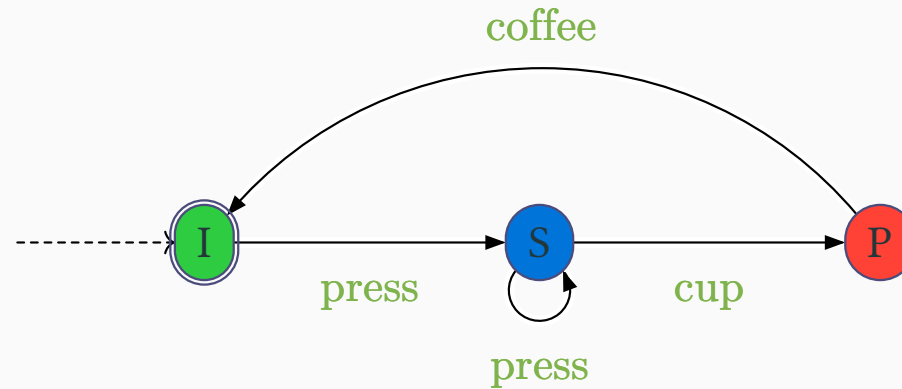


Example: The coffee machine



A finite automaton is composed of:

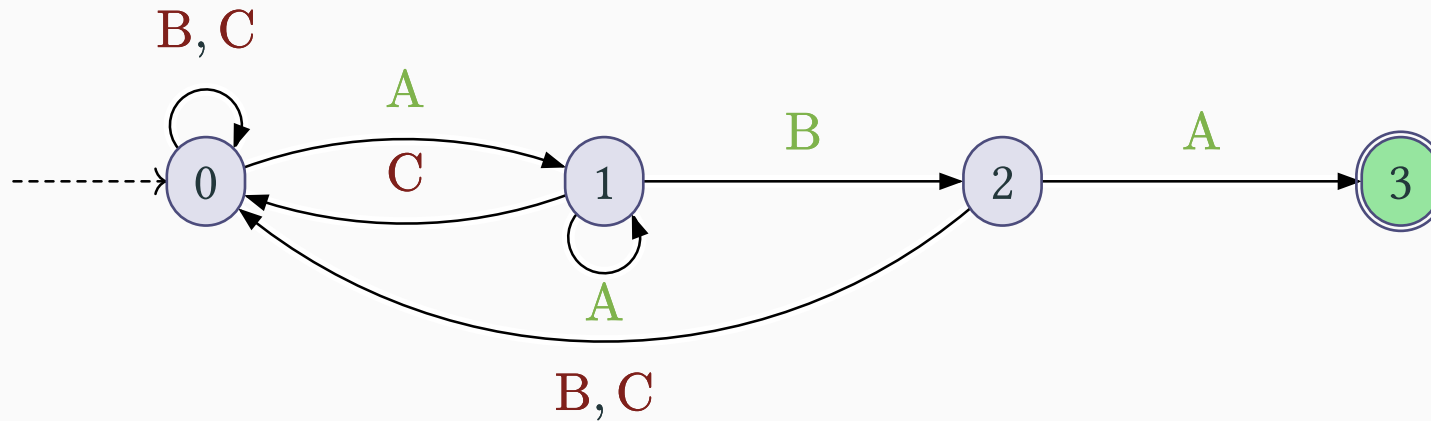
- ▶ a set of states
- ▶ an initial state
- ▶ a set of final states
- ▶ a transition relation
- ▶ an alphabet of actions



Example: The numerical code



- ▶ 3 keys A, B, C
- ▶ code to open door: **ABA**
- ▶ if the **wrong key** is pressed the whole operation has to **start again**



Exercise: The modulo 3 counter



- ▶ counter with 3 values: 0, 1, 2
- ▶ initial value: 0
- ▶ allows to **increment** and **decrement** the counter

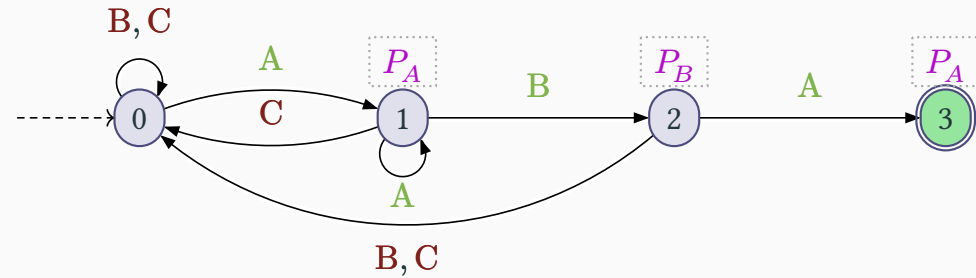
Atomic properties

- ▶ **Atomic properties** are elementary properties known to be true or false
- ▶ Atomic properties are properties that can be verified by looking at the **current state of the system**
 - ▶ e.g.: “the door is open”, “the coffee machine is in state *S*”
- ▶ Atomic properties are used to express **higher-level** properties of the system
 - ▶ e.g.: “the door is open when the key is pressed”
 - ▶ e.g.: “the coffee machine is in state *S* when the cup is full”



Numerical code atomic properties

- ▶ P_A : A has just been pressed (*correctly*)
- ▶ P_B : B has just been pressed (*correctly*)
- ▶ P_C : C has just been pressed (*correctly*)



Prove that the correct code was entered when the door opens

“Whenever the system is in state 3, then the last three keys pressed were ABA.”



Specification

- ▶ two processes execute and need access to the same resource
- ▶ each process can request access to a critical section of its code
- ▶ they must not execute this part at the same time
- ▶ when they have finished they signal they exit their critical section and loop
- ▶ back to their initial state

Questions

1. Model this problem with an automaton
2. Associate atomic properties with each state
3. Is the mutual exclusion requirement satisfied?
4. Is the system fair?
5. What would happen if you wanted to add a third process?

Solution: Mutual exclusion



Synchronization

- ▶ **independent** actions lead to a Cartesian product of states
- ▶ **synchronized** actions occur simultaneously

Cartesian product (formal definition)

Let $(\mathcal{A}_i)_{i \leq n}$ be a family of automata, $\mathcal{A}_i = (Q_i, \Sigma_i, q_{0_i}, F_i, \xrightarrow{i})$.

Definition 1 (Cartesian product of automata)

The **Cartesian product** of $(\mathcal{A}_i)_{i \leq n}$ is the automaton $\mathcal{A} = \mathcal{A}_1 \times \dots \times \mathcal{A}_n$ defined as follows:

- ▶ $Q = Q_1 \times \dots \times Q_n$
- ▶ $\Sigma = \prod_{\{1 \leq i \leq n\}} (\Sigma_i \cup \{\varepsilon\})$ (where ε represent a **silent action**)
- ▶ $q_0 = (q_{0_1}, \dots, q_{0_n})$
- ▶ $F = \{(q_1, \dots, q_n) \in Q \mid \exists i \in [1, n], q_i \in F_i\}$
- ▶ $\xrightarrow{} = \left\{ ((q_1, \dots, q_n), (a_1, \dots, a_n), (q'_1, \dots, q'_n)) \mid \forall i \in [1, n], (a_1 = \varepsilon \wedge q'_1 = q_1) \vee (a_1 \neq \varepsilon \wedge (q_1, a_1, q'_1) \in \xrightarrow{i}) \right\}$



Synchronized product (formal definition)

Let $(\mathcal{A}_i)_{i \leq n}$ be a family of automata, $\mathcal{A}_i = (Q_i, \Sigma_i, q_{0_i}, F_i, \xrightarrow{i})$.

Definition 2 (Synchronization set)

The **synchronization set**, denoted by Sync, describes all permitted simultaneous actions:

$$\text{Sync} \subseteq \prod_{1 \leq i \leq n} (\Sigma_i \cup \{\varepsilon\})$$



Definition 3 (Synchronized product of automata)

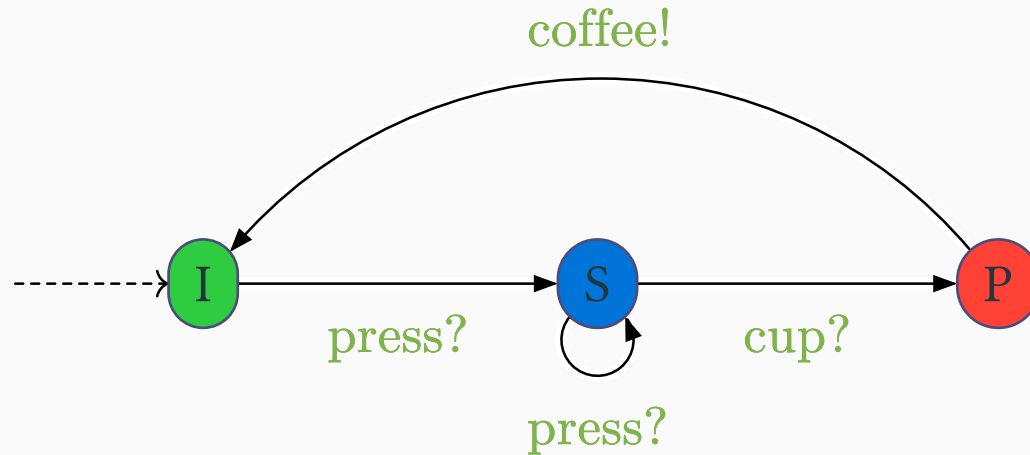
The **synchronized product** of $(\mathcal{A}_i)_{i \leq n}$ over a set Sync is the Cartesian product restricted to $\Sigma = \text{Sync}$.



Synchronization by message passing

Notations

- ▶ $m!$ **send** a message m
 - ▶ $m?$ **receive** a message m
-
- ▶ reception and send occur **simultaneously**
 - ▶ they concern the **same message**



Exercise. Is this coffee machine environment-unfriendly (providing disposable cups) or environment-friendly (requesting the user to bring their own cup)?



Model a lift in a 3-level building, made of:

the cabin which goes up and down according to the current level and the lift controller commands

3 doors (one per level) which open and close according to the controller's commands

a controller which operates the lift

Solution: A small lift





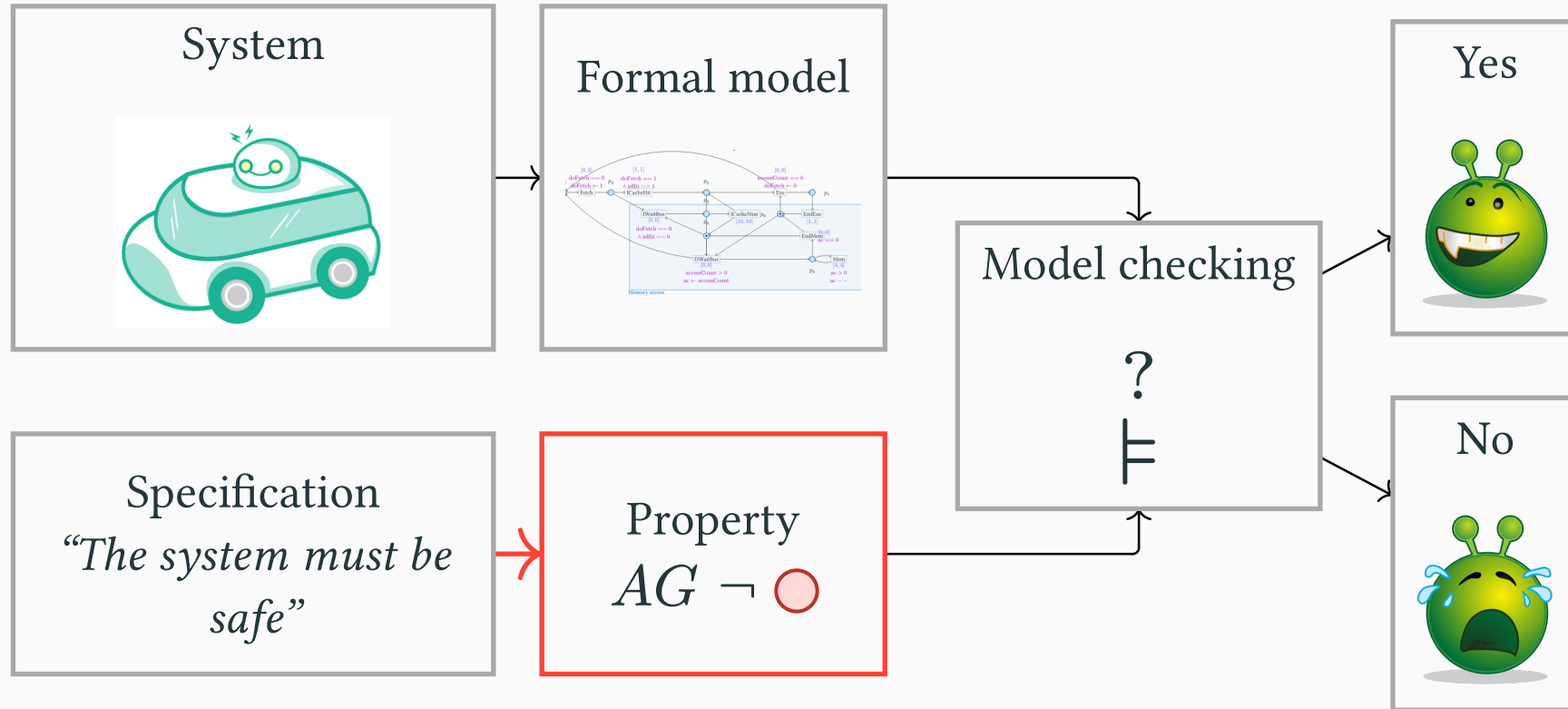
1. Model the mutual exclusion problem with message passing:
 - ▶ one automaton per participating process (2 processes)
 - ▶ a controller
2. How do you add a new process? Give the model for 3 processes, and explain how to generalize it to n processes

Solution: Mutual exclusion problem



Specifications

Modeling



Introduction

Introduction to temporal logics

- ▶ Express **dynamic behavior** of the system
- ▶ Use **formal syntax** and **semantics** to avoid any ambiguity
- ▶ Capture statements and reasoning that involve the notion of **order in time**

Example.

- ▶ “any request must ultimately be satisfied”
- ▶ “the lift never traverses a level for which a request is pending without satisfying the request”

Why temporal logics?

- ▶ We may use **first order logic** to express properties of the system
 - ▶ “*all requests will be satisfied one day*”
 - ▶ $\forall t \cdot (\text{request} \rightarrow \exists t' \geq t \cdot (\text{response}))$
- ▶ But we don't do that
 - ▶ Difficult to understand
 - ▶ Difficult to reason about
 - ▶ Difficult to implement

- ▶ **Temporal logics** are a family of logics
 - ▶ Allow to express properties over **sequences** of observations
 - ▶ Allow to reason about the **temporal order** of events
- ▶ Use of temporal operators and quantifiers over paths

Temporal $\neq$ Timed

(temporel $\neq$ temporisé)

- ▶ Temporal logics do not quantify the passage of time

Different types of properties

- ▶ **Invariance** (*invariance*): all states of the system satisfy a certain property
- ▶ **Safety** (*sûreté*): something bad never happens
- ▶ **Accessibility** (*accessibilité*): a given state is reachable from the initial state
- ▶ **Liveness** (*vivacité*): something “good” will eventually happen
- ▶ **Fairness** (*équité*): something will happen infinitely often

Linear-time Temporal Logic (LTL)

- ▶ Properties are expressed over **linear sequences** of states

Computation Tree Logic (CTL)

- ▶ Properties are expressed over **trees** of states
- ▶ Allows to express properties over **branches** of the tree

LTL



LTL expresses formulas on the **order** between the **future** atomic propositions **for one given path**, over a set of atomic propositions **AP**

Definition 1 (Minimal syntax of LTL)

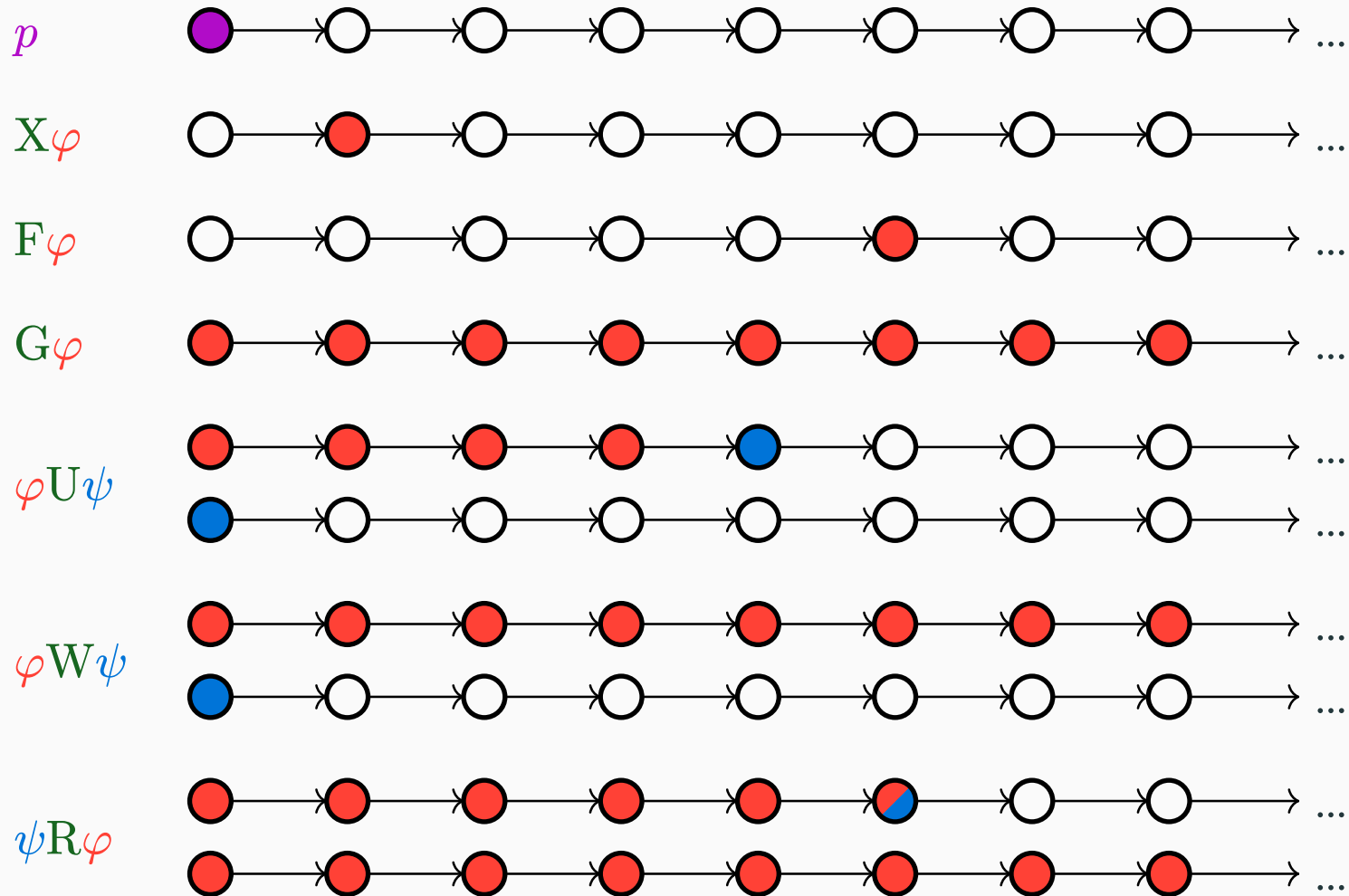
$\text{LTL} \ni \varphi ::= p \mid \neg \varphi \mid \varphi \wedge \varphi \mid X\varphi \mid \varphi U \psi$



- ▶ Additional operators can be defined

Name	Definition
Eventually	$\mathbf{F}\varphi \equiv \top \mathbf{U} \varphi$
Globally	$\mathbf{G}\varphi \equiv \neg \mathbf{F} \neg \varphi$
Weak until	▶ $\varphi \mathbf{W} \psi \equiv (\varphi \mathbf{U} \psi) \vee \mathbf{G}\varphi$ ▶ $\mathbf{W}$ is similar to $\mathbf{U}$ but ψ may never happen
Release	▶ $\psi \mathbf{R} \varphi \equiv \neg(\neg \psi \mathbf{U} \neg \varphi)$ ▶ ψ releases φ: φ must hold until and including the point where ψ holds

Illustration of LTL semantics



- ▶ Let ρ be a finite run and $p \in \text{AP}$ an atomic proposition.
- ▶ “ $\rho, i \models \varphi$ ” denotes that, at position i of its execution, ρ satisfies formula φ .

Definition 2 (Semantics of LTL)

$\rho, i \models p$	if $p \in \text{lab}(\rho(i))$
$\rho, i \models \neg\varphi$	if $\rho, i \not\models \varphi$
$\rho, i \models \varphi \wedge \psi$	if $\rho, i \models \varphi$ and $\rho, i \models \psi$
$\rho, i \models \text{X}\varphi$	if $i < \rho $ and $\rho, i + 1 \models \varphi$
$\rho, i \models \text{F}\varphi$	if $\exists j$ s.t. $i \leq j \leq \rho : \rho, j \models \varphi$
$\rho, i \models \text{G}\varphi$	if $\forall j$ s.t. $i \leq j \leq \rho : \rho, j \models \varphi$
$\rho, i \models \varphi \text{U} \psi$	if $\exists j$ s.t. $i \leq j \leq \rho : \rho, j \models \psi$ and $\forall k$ s.t. $i \leq k < j : \rho, k \models \varphi$





Express $\vee$, $\implies$, $\iff$ by using $\neg$ and $\wedge$ only.

$$\varphi \vee \psi \equiv$$

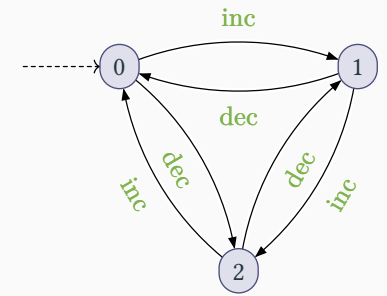
$$\varphi \implies \psi \equiv$$

$$\varphi \iff \psi \equiv$$



- ▶ What do the following formulae mean?
- ▶ Which runs satisfy the LTL property?

1. $\text{XXX}0$
2. $\text{F}(1 \vee 2)$
3. $\text{F}1$

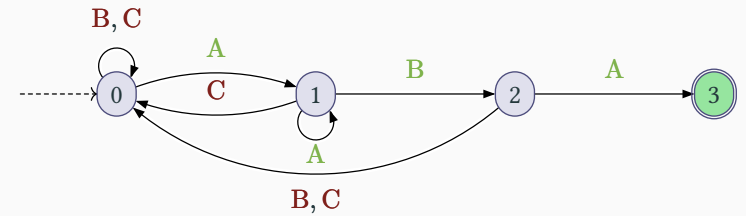




- ▶ What do the following formulae mean?
- ▶ Which runs satisfy the LTL property?

1. $F3$

2. $G\neg 3$



Exercise: Writing LTL formulae (1)



▶ The property p will be true one day ($\text{AP} = \{p\}$)	
▶ The property p is always true	
▶ The property p is true at step 1	
▶ The property p is stable (if it happens, it remains true)	
▶ Every request will eventually be satisfied ($\text{AP} = \{\text{request}, \text{grant}\}$)	
▶ Every request will eventually be satisfied and no additional request occurs before this request is satisfied ($\text{AP} = \{\text{request}, \text{grant}\}$)	
▶ Every time money is withdrawn, the correct PIN was provided before ($\text{AP} = \{\text{withdraw}, \text{pin_ok}\}$)	

Exercise: Writing LTL formulae (1)



▶ Once the light is green, it cannot turn red immediately after ($AP = \{\text{red}, \text{green}\}$)	
▶ Every time the light is red, it will turn green one day in the future ($AP = \{\text{red}, \text{green}\}$)	
▶ When the light is green, it will turn red after being orange ($AP = \{\text{red}, \text{green}, \text{orange}\}$)	
▶ When the light is green, it will only turn red after being orange ($AP = \{\text{red}, \text{green}, \text{orange}\}$)	



We consider the following trace prefix: $\emptyset, \{p\}, \{p, q\}, \{p\}, \{p\}, \emptyset, \{p, q\}$

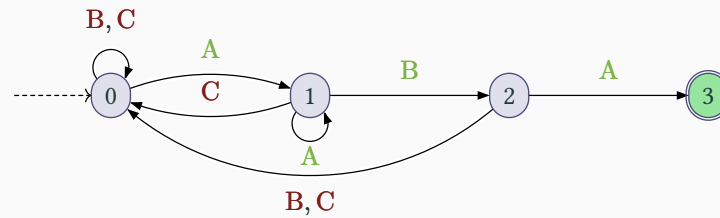
Does the following formulae hold at each position of this trace prefix?

	$\emptyset$	$\{p\}$	$\{p, q\}$	$\{p\}$	$\{p\}$	$\emptyset$	$\{p, q\}$
$p \wedge q$							
$\mathbf{F}(p \wedge q)$							
$p \mathbf{U} q$							
$\mathbf{F}p \wedge \mathbf{F}q$							
$\mathbf{G}(p \vee q)$							



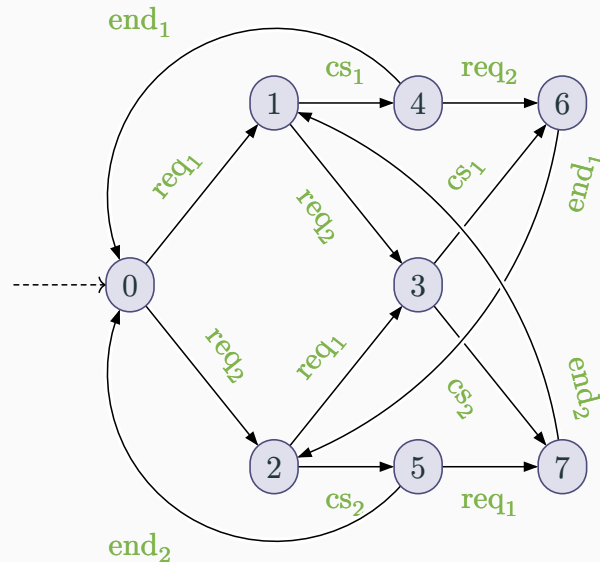
▶ “The plane will never crash”	
▶ “I will eventually get a job”	
▶ “If I ask a question, the teacher will answer me”	
▶ “If I ask for food infinitely often, then I will get food infinitely often”	

Exercise: Writing LTL formulae (3)



1. Write an LTL formula satisfied by all runs where keys A and B have successively been pressed
2. Write an LTL formula that characterizes the infinite loop on state 0

Exercise: Writing LTL formulae (4)



Exercise. Write an LTL formula satisfied by all runs where:

1. The two processes are not simultaneously in the critical section
2. Whenever process 1 requests to enter the critical section, it will eventually succeed

R_1	1, 3, 7	R_2	2, 3, 6
CS_1	4, 6	CS_2	5, 7
I_1	0, 2, 5	I_2	0, 1, 4



Using the aforementioned formal semantics, prove that:

1. $\mathbf{F}\varphi \equiv \top \mathbf{U} \varphi$
2. $\mathbf{G}\varphi \equiv \neg \mathbf{F} \neg \varphi$

Exercise: LTL Equivalences (2)



Does the following equivalences hold? If so, prove it, otherwise give a counter-example trace.

▶ $G(Fp \wedge Fq)$ and $GFp \wedge GFq$	
▶ $F(Gp \wedge Gq)$ and $FGp \wedge FGq$	
▶ $G(Fp \vee Fq)$ and $GFp \vee GFq$	
▶ $F(Gp \vee Gq)$ and $FGp \vee FGq$	
▶ $GF(p \wedge q)$ and $GFp \wedge GFq$	
▶ $GF(p \vee q)$ and $GFp \vee GFq$	
▶ $FG(p \wedge q)$ and $FGp \wedge FGq$	
▶ $FG(p \vee q)$ and $FGp \vee FGq$	

CTL



CTL expresses formulas on the **order** between the **future** atomic propositions **for some or for all paths**, over a set of atomic propositions **AP**

Definition 1 (Minimal syntax of CTL)

$\text{CTL} \ni \varphi ::= p \mid \neg \varphi \mid \varphi \vee \varphi \mid \text{EX} \varphi \mid \text{E} \varphi \text{U} \psi \mid \text{A} \varphi \text{U} \psi$



- ▶ Additional operators can be defined

Name	Definition
Always next	
Exists eventually	
Always eventually	
Exists globally	
Always globally	
Exists weak until	

Same as LTL, but with paths quantified by **A** or **E**:

Definition 2 ((partial) Semantics of CTL)

$\rho, i \models \mathbf{E}\varphi$	if $\exists \rho' : \rho(0) \dots \rho(i) = \rho'(0) \dots \rho'(i)$ and $\rho', i \models \varphi$
$\rho, i \models \mathbf{A}\varphi$	if $\forall \rho'$ s.t. $\rho(0) \dots \rho(i) = \rho'(0) \dots \rho'(i)$ we have $\rho', i \models \varphi$



In CTL, each use of a temporal operator (**X**, **F**, **G**, **R**, **U**) must be in the **immediate scope** of a quantifier (**E**, **A**)¹

¹CTL* allows this constraint to be released

CTL: alternating quantifiers

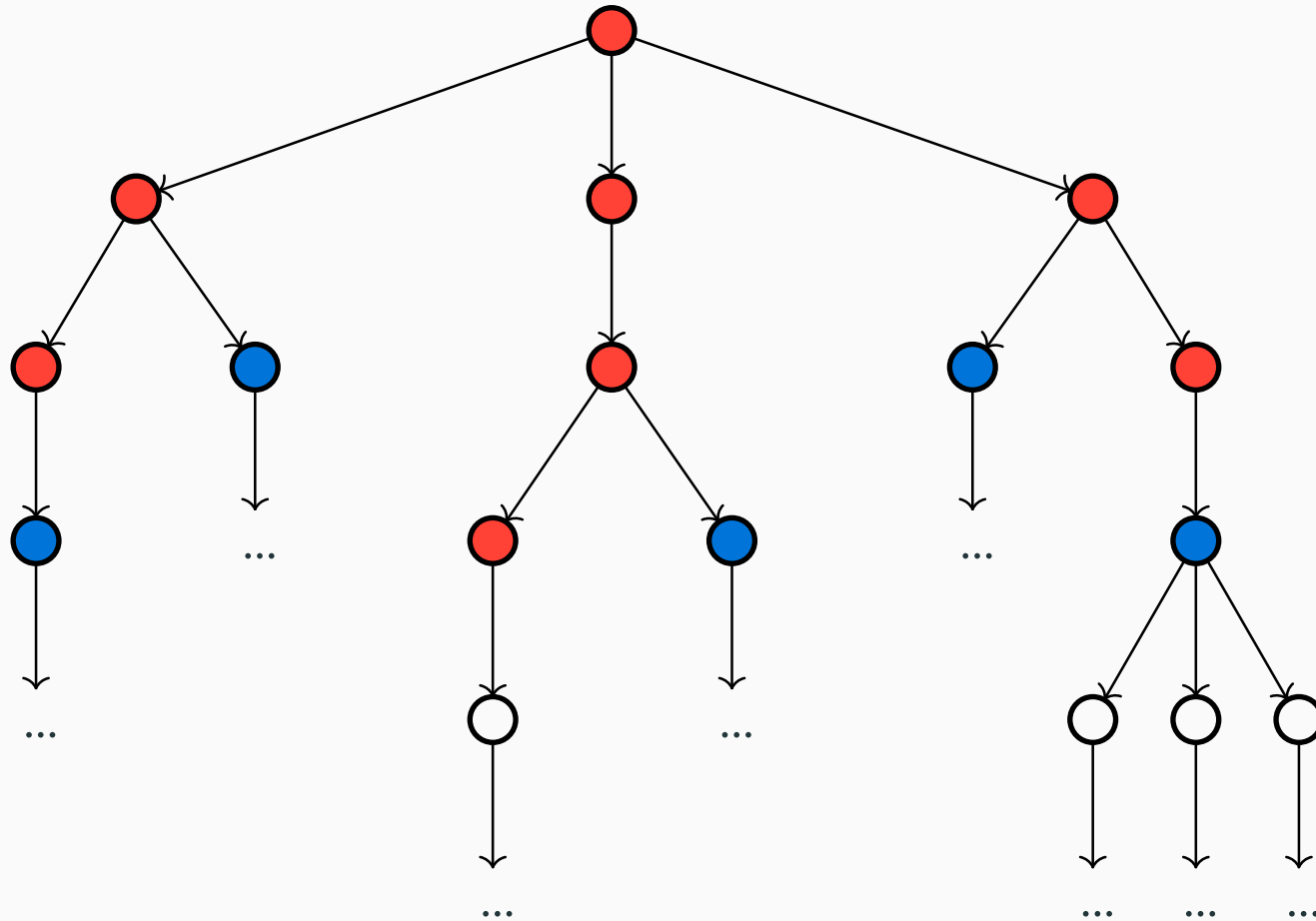
A path quantifier must always be followed by a temporal operator

Some useful combinations:

“There exists a path for which the next state is...”	EX
“For all possible paths, the next state is...”	AX
“It is possible that eventually...”	EF
“In any case, eventually...”	AF
“In any case, for all states...”	AG

Illustration of CTL semantics

$A\varphi U\psi$

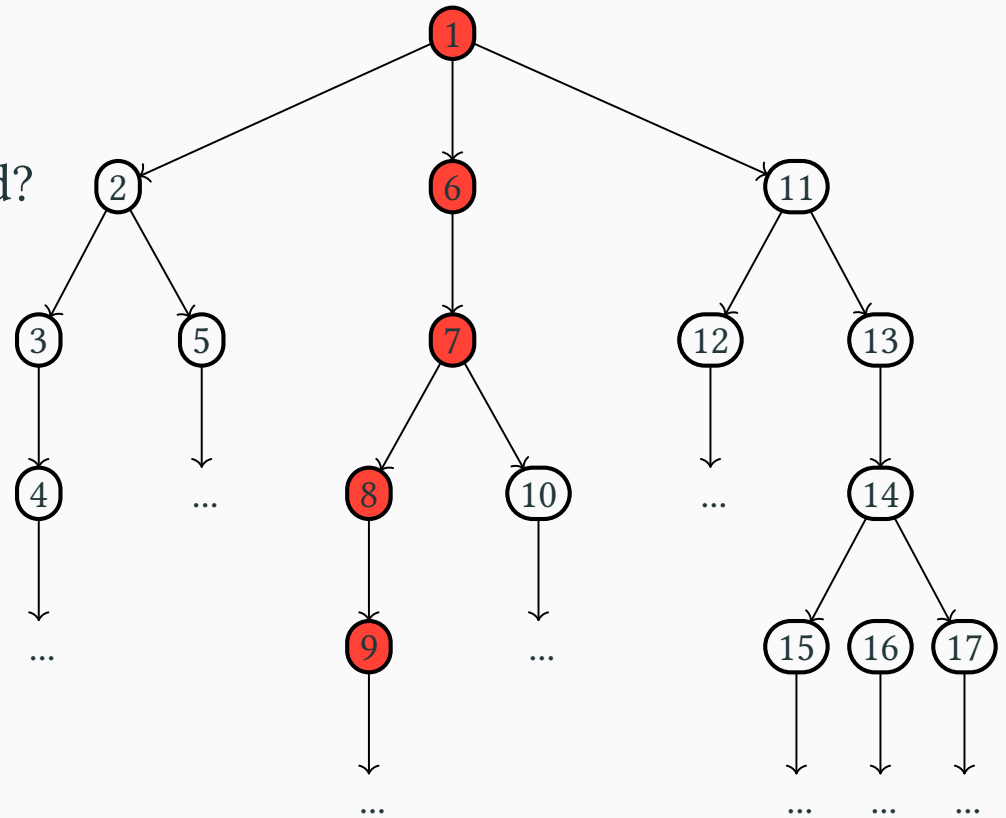


Exercise: Illustration of the CTL semantics



On which states are the following formulae valid?

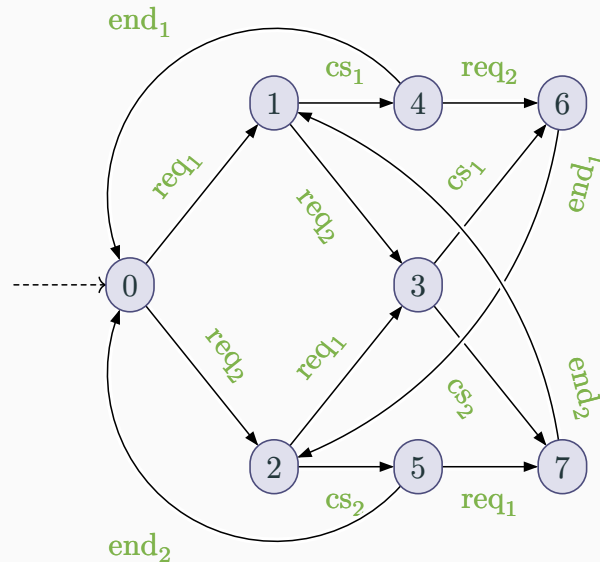
1. $EX\varphi$
2. $EF\varphi$
3. $EG\varphi$
4. $AX\varphi$
5. $AG\varphi$





▶ “Whatever happens, the plane will never crash”	
▶ “Whatever happens, I will eventually get a job”	
▶ “I may eventually get a job”	
▶ “I may study for the rest of my life (and beyond)”	
▶ “It can always happen that suddenly I discover formal methods and then I may use them for the rest of time”	

Exercise: Mutual exclusion



Express the meaning of the following properties and indicate whether they are satisfied.

1. $AG \neg (CS_1 \wedge CS_2)$
2. $AGR_1 \implies AF CS_1$
3. $AG(EF(I_1 \wedge I_2))$

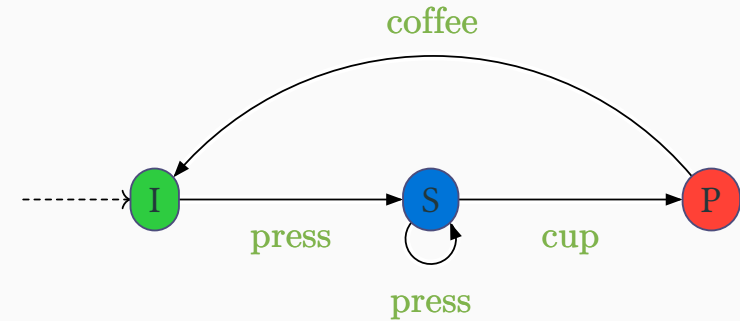
R_1	1, 3, 7	R_2	2, 3, 6
CS_1	4, 6	CS_2	5, 7
I_1	0, 2, 5	I_2	0, 1, 4



Express in CTL the following properties, and decide whether they are satisfied for the coffee machine

We reason here with actions instead of state predicate

- ▶ “At any time, if the button is pressed, a coffee is always eventually delivered”
- ▶ “At any time, if the button is pressed, there exists an execution such that a coffee is eventually delivered”
- ▶ “Once the cup is placed, coffee will necessarily come next”
- ▶ “It is possible to get a coffee with exactly 2 doses of sugar”



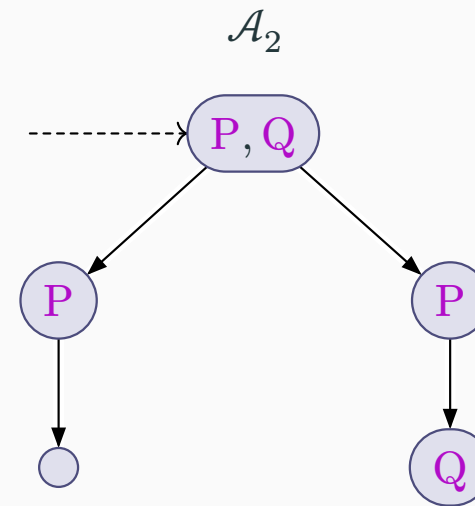
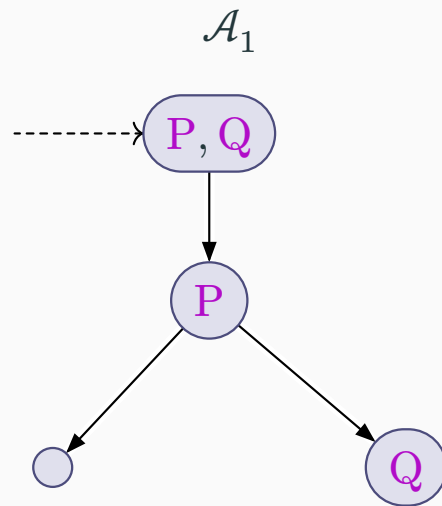


Using the formal semantics of CTL, prove that:

1. $EF\varphi \equiv E\top U\varphi$
2. $AX\varphi \equiv \neg EX\neg\varphi$
3. $AF\varphi \equiv \neg EG\neg\varphi$
4. $AG\varphi \equiv \neg(E\top U\neg\varphi)$

LTL vs. CTL

Not the same behaviors



LTL

- ▶ Runs for both automata
 - ▶ $\{P, Q\}\{P\}\{-\}$
 - ▶ $\{P, Q\}\{P\}\{Q\}$

$$\forall \varphi : \mathcal{A}_1 \models \varphi \iff \mathcal{A}_2 \models \varphi$$

CTL

$$\mathcal{A}_1 \models \text{AX}(\text{EX}Q \wedge \text{EX}\neg Q)$$

$$\mathcal{A}_1 \not\models \text{AX}(\text{EX}Q \wedge \text{EX}\neg Q)$$



▶ $(P \cup Q) \vee GP$	
▶ GFP	
▶ $AG(P \Rightarrow AFQ)$	
▶ $A(GFP \Rightarrow AGQ)$	

This presentation can be published, reused and modified under the terms of the license Creative Commons **Attribution-NonCommercial-ShareAlike 4.0 International (CC BY-NC-SA 4.0)**



creativecommons.org/licenses/by-nc-sa/4.0/

Author: **Dylan Marinho**

*Inspired by the courses of Étienne André and
Nathalie Sznajder*

